

04.3 — Disaster Recovery Runbook

For: Recovering from total env loss — destroyed container, wiped host, regional outage, accidental down -v. **Goal:** Restore prod from offsite backup within RTO (1 hour). **Last updated:** 2026-08-29 **Pre-req reading:** 00.2 Architecture Overview, 04.1 Deployment, 04.2 Daily Ops, 04.4 Incident Response

Purpose

Recover Haritha Hospitals ERPNext from catastrophic loss using the 3-2-1 backup strategy documented in MEMORY.md. Covers three scenarios:

1. **Single container loss** — erp-prod-backend-1 or erp-prod-db-1 corrupt/wiped; other containers intact
2. **Single host loss** — main VPS 144.217.163.228 down; offsite backup intact
3. **Total loss** — both main VPS + venkat VPS down; only GitHub repos + offsite cold storage intact

Recovery time targets:

Tier	RTO (Recovery Time Objective)	RPO (Recovery Point Objective)
Single container	15 minutes	0 (named volumes preserved)
Single host	1 hour	6 hours (last backup slot)
Total loss	4 hours	6 hours (last backup slot)

RTO / RPO targets

Scenario	RTO	RPO	Rationale
Container crash, data intact	15 min	0	docker restart only — no restore needed
Container data wipe	30 min	6 hours	restore from latest backup slot
Host down (VPS provider failure)	1 hour	6 hours	provision new VPS + restore
Region-wide outage (both VPSes down)	4 hours	6 hours	rebuild from cold storage + GitHub

Why 6-hour RPO: backup cron runs every 6 hours (0 */6 * * * = 00/06/12/18 IST). Worst case data loss = 6 hours of attendance/leave/payroll records entered after the last successful backup slot.

Why 1-hour RTO: offsite backup is hot (rsync'd immediately after creation), new VPS can be provisioned in 5-10 min on DigitalOcean/Linode, restore + bring-up = ~30 min.

Backup locations (3-2-1 rule)

Location	Retention	Type	Path
Local (main VPS)	7 days	tar.gz + .sha256	/home/vijay/backups/{prod,qa,d
Offsite (venkat VPS)	forever	tar.gz + .sha256	venkat@135.125.196.35:/home/ve
Cold (git remote)	forever	code only	git@github.com:venkat-narasimha/haritha_hospital.git

3-2-1 compliance: - 3 copies (local + offsite + code in git) - 2 media types (disk + git) - 1 offsite (venkat VPS in different region)

Note: **dev-erp reverses the direction** — local = forever on venkat VPS, offsite = 7d on main VPS. Both directions are valid 3-2-1.

Backup scripts (Lesson #79 hardened)

Script	Path	Schedule	Hardening
pberpprod_backup.sh	/home/vijay/scripts/pberpprod_backup.sh	0 0 1 * * *	Lesson #79 (timeout 900, set -euo pipefail, \${PIPESTATUS[0]} capture)
prod_backup.sh	/home/vijay/scripts/prod_backup.sh	0 0 1 * * *	verify if still in cron (deprecated)
dev_backup.sh	/home/vijay/scripts/dev_backup.sh	0 0 1 * * *	pre-Lesson #79 pattern — should harden
qa_backup.sh	/home/vijay/scripts/qa_backup.sh	0 0 1 * * *	pre-Lesson #79 pattern — should harden
deverp_backup.sh	/home/venkat/scripts/deverp_backup.sh	0 0 1 * * *	mirrors dev_backup.sh, also pre-hardening (on venkat VPS)

△ **Known gap:** dev_backup.sh, qa_backup.sh, and deverp_backup.sh may still be pre-Lesson #79 (no set -euo pipefail, no timeout 900). They run, but on edge-case errors (silent glob miss, hung bench) could fail invisibly. **Action: harden these scripts in next quarterly maintenance window.**

Backup bundle format (2026-08-27 fix)

Each backup is ONE tar.gz containing 4 files:

```
pberpprod_backup_20260829_030000.tar.gz
├── 20260829_030000-pberpprod_duckdns_org-database.sql.gz      # Frappe DB dump
├── 20260829_030000-pberpprod_duckdns_org-site_config_backup.json # site_config.json snapshot
├── 20260829_030000-pberpprod_duckdns_org-files.tar            # public/files (attachments)
└── 20260829_030000-pberpprod_duckdns_org-private-files.tar    # private/files (private attachments)
```

Plus .sha256 sidecar for integrity check.

Cron schedule

```
# Main VPS (vijay)
0 */6 * * * /home/vijay/scripts/pberpprod_backup.sh >> /home/vijay/backups/logs/pberpprod_ba
0 */6 * * * /home/vijay/scripts/qa_backup.sh >> /home/vijay/backups/logs/qa_backup_cron
0 */6 * * * /home/vijay/scripts/dev_backup.sh >> /home/vijay/backups/logs/dev_backup_cron
```

```
# venkat VPS (deverp)
0 */6 * * * /home/venkat/scripts/deverp_backup.sh >> /home/venkat/backups/logs/deverp_backup
```

Slots fire at **00:00, 06:00, 12:00, 18:00 IST**. Expect 1-3 min per env per slot.

Verify cron is intact: see 04.2 Daily Ops §“Cron line integrity”.

Full prod rebuild procedure

Scenario: main VPS is destroyed (provider failure, accidental wipe). Need to bring pberpprod back from offsite backup.

Step 1 — Provision new VPS (5-10 min)

```
# Choose provider (DigitalOcean / Linode / Hetzner recommended – all < 10 min provision)
# Minimum spec: 4 vCPU, 8 GB RAM, 80 GB SSD, Ubuntu 22.04 LTS
# Notes:
# - Same region as venkat VPS (lower latency for rsync)
# - Same SSH key installed
# - Same hostname (or update MEMORY.md)
```

Step 2 — Install Docker + base tools (3 min)

```
ssh -i /root/.openclaw/ssh_key root@<new-vps-ip>
```

```
# Install Docker
curl -fsSL https://get.docker.com -o get-docker.sh
sh get-docker.sh
```

```
# Install useful tools
apt-get install -y jq rsync curl openssl
```

```
# Create user vijay (if not auto-created)
useradd -m -s /bash vijay
mkdir -p /home/vijay/.ssh
cp /root/.ssh/authorized_keys /home/vijay/.ssh/ # or paste pubkey
chown -R vijay:vijay /home/vijay/.ssh
```

Step 3 — Clone repos (2 min)

```
# ERPClaw repo (compose files, env templates)
ssh vijay@<new-vps-ip>
cd /home/vijay
```

```
git clone git@github.com:venkat-narasimha/erpclaw.git frappeclaw

# Haritha custom app
cd /home/vijay/frappeclaw/frappeclaw-data/workspace/projects/
git clone git@github.com:venkat-narasimha/haritha_hospital.git haritha-hospitals
```

Step 4 — Recreate compose stack (5 min)

```
cd /home/vijay/frappeclaw

# Copy env files (may need to recreate from template + MEMORY.md secrets)
# If env files lost: copy from git history, OR pull from backup (Lesson #5: secrets in .env)
#   → if no backup, must reset ALL passwords + notify Venkat

# Bring up stack
docker compose -f frappeclaw-compose.yml up -d

# Wait for healthy
docker compose -f frappeclaw-compose.yml ps
# Expect: all "Up" + "healthy" within 60-90 seconds
```

Step 5 — Restore from offsite backup (10 min)

```
# 1. Identify latest prod backup
LATEST=$(ssh -i /root/.openclaw/venkat_vps_key venkat@135.125.196.35 \
  "ls -t /home/venkat/pberpprod_backups/*.tar.gz | head -1" | tr -d '\r')
echo "Restoring from: $LATEST"

# 2. Download to local temp
mkdir -p /tmp/restore
scp -i /root/.openclaw/venkat_vps_key venkat@135.125.196.35:$LATEST /tmp/restore/

# 3. Verify integrity
sha256sum /tmp/restore/*.tar.gz
ssh -i /root/.openclaw/venkat_vps_key venkat@135.125.196.35 \
  "sha256sum $LATEST" # compare outputs

# 4. Extract
cd /tmp/restore
tar -xzf *.tar.gz

# 5. Identify the inner files (Lesson #79 bundle format)
ls
# Expect 4 files matching the timestamp prefix:
# 20260829_030000-pberpprod_duckdns_org-database.sql.gz
# 20260829_030000-pberpprod_duckdns_org-site_config_backup.json
# 20260829_030000-pberpprod_duckdns_org-files.tar
# 20260829_030000-pberpprod_duckdns_org-private-files.tar
```

Step 6 — Copy into container (3 min)

```
SITE=pberpprod.duckdns.org
TIMESTAMP=$(ls *-database.sql.gz | head -1 | cut -c1-15)
```

```

docker cp ${TIMESTAMP}-${SITE}-database.sql.gz erp-prod-backend-1:/tmp/
docker cp ${TIMESTAMP}-${SITE}-site_config_backup.json erp-prod-backend-1:/tmp/
docker cp ${TIMESTAMP}-${SITE}-files.tar erp-prod-backend-1:/tmp/
docker cp ${TIMESTAMP}-${SITE}-private-files.tar erp-prod-backend-1:/tmp/

```

Step 7 — Restore DB (5 min)

```

# 1. Stop bench to prevent writes during restore
docker exec erp-prod-backend-1 bash -c '
    supervisorctl stop frappe-bench-workers: frappe-bench-web: frappe-bench-schedule:'

# 2. Get DB credentials
DB_PASS=$(docker exec erp-prod-db-1 printenv MYSQL_ROOT_PASSWORD)
DB_NAME=$(docker exec erp-prod-db-1 printenv MARIADB_DATABASE)
echo "Restoring to DB: $DB_NAME"

# 3. Drop + recreate DB (CAREFUL – wipes any existing data)
docker exec erp-prod-db-1 mariadb -u root -p"$DB_PASS" -e "
    DROP DATABASE IF EXISTS \`$DB_NAME\`;
    CREATE DATABASE \`$DB_NAME\`;"

# 4. Restore SQL
docker exec erp-prod-backend-1 bash -c "
    gunzip -c /tmp/${TIMESTAMP}-${SITE}-database.sql.gz | \
    mysql -h erp-prod-db-1 -u root -p"$DB_PASS" $DB_NAME"
# Note: piping through backend avoids needing mysql client in db container

# 5. Restore site_config
docker exec erp-prod-backend-1 bash -c "
    cp /tmp/${TIMESTAMP}-${SITE}-site_config_backup.json \
    /home/frappe/frappe-bench/sites/${SITE}/site_config.json"

```

Step 8 — Restore files (5 min)

```

# 1. Wipe existing (CAREFUL – assumes backup is canonical)
docker exec erp-prod-backend-1 bash -c "
    rm -rf /home/frappe/frappe-bench/sites/${SITE}/public/files/* &&
    rm -rf /home/frappe/frappe-bench/sites/${SITE}/private/files/*"

# 2. Extract public files
docker exec erp-prod-backend-1 bash -c "
    tar -xf /tmp/${TIMESTAMP}-${SITE}-files.tar \
    -C /home/frappe/frappe-bench/sites/${SITE}/public/"

# 3. Extract private files
docker exec erp-prod-backend-1 bash -c "
    tar -xf /tmp/${TIMESTAMP}-${SITE}-private-files.tar \
    -C /home/frappe/frappe-bench/sites/${SITE}/private/"

# 4. Fix permissions
docker exec -u root erp-prod-backend-1 bash -c "
    chown -R frappe:frappe /home/frappe/frappe-bench/sites/${SITE}/public/files &&
    chown -R frappe:frappe /home/frappe/frappe-bench/sites/${SITE}/private/files"

```

Step 9 — Install haritha_hospital app (2 min)

```
# 1. Get app (already cloned to /home/vijay/frappeclaw-data/workspace/projects/haritha-hos
# But container needs it inside bench/apps/

# Option A: copy from workspace to container
docker exec erp-prod-backend-1 bash -c '
  mkdir -p /home/frappe/frappe-bench/apps/haritha_hospital'
docker cp /home/vijay/frappeclaw/frappeclaw-data/workspace/projects/haritha-hospitals/. \
  erp-prod-backend-1:/home/frappe/frappe-bench/apps/haritha_hospital/

# 2. Install (Lesson #151 requires hooks.py fixtures declaration)
docker exec erp-prod-backend-1 bash -c '
  cd /home/frappe/frappe-bench &&
  bench --site pberpprod.duckdns.org install-app haritha_hospital'

# 3. CRITICAL: restart backend (Lesson #153 – gunicorn --preload)
docker restart erp-prod-backend-1
sleep 45
```

Step 10 — Restart bench + verify (3 min)

```
# 1. Start workers/web/scheduler
docker exec erp-prod-backend-1 bash -c '
  supervisorctl start frappe-bench-workers: frappe-bench-web: frappe-bench-schedule:'

# 2. Verify
curl -sS -o /dev/null -w "%{http_code}\n" https://pberpPROD.duckdns.org/ # expect 302
# 3. Login test (use credentials from MEMORY.md or Venkat)
curl -sS -c /tmp/c.txt -X POST https://pberpPROD.duckdns.org/api/method/login \
  -H 'Content-Type: application/json' \
  -d '{"usr": "Administrator", "pwd": "<from MEMORY>"}' | jq .

# 4. Verify custom fields load
curl -sS -b /tmp/c.txt https://pberpPROD.duckdns.org/api/resource/Custom\ Field/Employee-c
  -H 'Accept: application/json' | jq .data.module # expect "haritha_hospital"
```

Step 11 — Update DNS + cert (5 min)

```
# 1. Update DuckDNS to point to new VPS IP
curl -k 'https://www.duckdns.org/update?domains=pberpPROD&token=<DUCKDNS_TOKEN>&ip=<NEW_VP

# 2. Force cert renewal on new VPS
ssh vijay@<new-vps-ip> 'docker exec nginx-proxy /app/force-renew'

# 3. Verify cert
echo | openssl s_client -servername pberpPROD.duckdns.org \
  -connect pberpPROD.duckdns.org:443 2>/dev/null | \
  openssl x509 -noout -enddate
```

Step 12 — Re-establish cron + offsite (5 min)

```
# Re-add backup cron (Lesson #77 atomic pattern)
ssh vijay@<new-vps-ip> 'crontab -l > /tmp/cron.bak'
```

```

cat >> /tmp/cron.bak <<'EOF'
0 */6 * * * /home/vijay/scripts/pberpprod_backup.sh >> /home/vijay/backups/logs/pberpprod_
EOF
ssh vijay@<new-vps-ip> 'crontab /tmp/cron.bak'

# Verify offsite push works (one-shot run)
ssh vijay@<new-vps-ip> '/home/vijay/scripts/pberpprod_backup.sh'
ssh -i /root/.openclaw/venkat_vps_key venkat@135.125.196.35 \
  "ls -lt /home/venkat/pberpprod_backups/ | head -3" # confirm new bundle present

```

Total time: ~45-60 min

Partial recovery — single container, data intact

Scenario: one container corrupted (e.g., backend won't start, DB container disk-full), but named volumes preserved.

```

# 1. Identify the issue
docker logs --tail 100 erp-prod-backend-1
docker inspect erp-prod-backend-1 | jq '.[0] | {State, Mounts}'

# 2. If just a hung process: restart
docker restart erp-prod-backend-1
sleep 45
curl -sS -o /dev/null -w "%{http_code}\n" https://pberpPROD.duckdns.org/

# 3. If data corruption in db container: restore DB only (not full backup)
#   → identify latest backup, restore just the SQL.gz (skip site_config + files)
#   → see Step 7 above

# 4. If public/private files lost: extract just those
#   → see Step 8 above

# 5. NEVER use `docker-compose down -v` (Lesson #154: deletes named volumes)

```

Partial recovery — single table

Scenario: accidentally truncated a table, want to restore just that table.

```

# 1. Find the latest backup containing the table
LATEST=$(ssh vijay@144.217.163.228 "ls -t /home/vijay/backups/prod/*.tar.gz | head -1")

# 2. Extract
mkdir -p /tmp/partial
ssh vijay@144.217.163.228 "tar -xzf $LATEST -C /tmp/partial"

# 3. Decompress DB
gunzip /tmp/partial/*-database.sql.gz

# 4. Find the table dump
SQL=/tmp/partial/*-database.sql

```

```

grep -n "CREATE TABLE.*\`tabAttendance\`" $SQL
# → get line range

# 5. Stop bench, drop the affected table, restore from dump slice
docker exec erp-prod-backend-1 bash -c '
    supervisorctl stop frappe-bench-workers: frappe-bench-web: frappe-bench-schedule:'

# Extract just that table's data (manual sed/awk)
# Then load via mysql client
docker exec erp-prod-db-1 mariadb -u root -p"$DB_PASS" -D _b80f05e76a0dcaad \
    -e "DROP TABLE IF EXISTS \`tabAttendance\`;"
    $(awk '/-- Table structure for table .tabAttendance./,/-- Table structure for table/'

# Restart bench
docker exec erp-prod-backend-1 bash -c '
    supervisorctl start frappe-bench-workers: frappe-bench-web: frappe-bench-schedule:'

```

This is finicky and risky. **For prod, prefer full backup restore over single-table restore** unless you really know what you're doing.

Test cadence — DR drill quarterly

Per Lesson #48 + #114:

- **Frequency:** every 90 days
- **Scope:** full backup restore to a scratch env (not prod)
- **Pass criteria:**
 - ☐ Latest backup downloads + integrity check passes
 - ☐ Restore completes within 60 min
 - ☐ Custom app installs + fixtures load
 - ☐ Login works
 - ☐ Sample records (Attendance, Shift Assignment) queryable
 - ☐ Custom Field Employee-custom_test shows correct module
- **Last DR drill:** *not yet performed* (placeholder — schedule for next maintenance window)
- **Next DR drill:** 2026-Q4 (target: 2026-11-15)

If a DR drill reveals gaps, add a new lesson to LEARNINGS.md and update this runbook.

Communication plan

During DR (live)

1. STOP all changes (notify Venkat: "DR in progress, no deploys until cleared")
2. Post status to memory/YYYY-MM-DD.md every 30 min:
 - what step you're on
 - what's verified
 - what's still uncertain
3. If RTO at risk (60 min approaching): notify Venkat IMMEDIATELY
4. If data loss discovered: notify Venkat IMMEDIATELY (don't continue silently)

After DR (post-mortem)

Within 24 hours:

- ☐ Update LEARNINGS.md with new lessons
 - ☐ Update this runbook with any gaps found
 - ☐ Notify Venkat: "DR complete. {time-to-recover}. {data-loss-if-any}."
 - ☐ If RTO breached: schedule retro to identify root cause
-

Cold-storage-only recovery (total loss)

If both VPSes are gone, only GitHub repos + offsite cold storage survive.

- # 1. Provision new VPS (same as Step 1 above)*
- # 2. Clone repos (same as Step 3)*
- # 3. Get backup from cold storage (e.g., S3 Glacier, Backblaze B2, physical disk)*
- # 4. Same Steps 5-12 as full prod rebuild*
- # 5. RTO may exceed 4 hours due to cold storage retrieval time*

Cold storage details: *not yet configured* (placeholder). Recommend setting up Backblaze B2 with weekly export of latest backup bundle, encrypted with age/gpg. Open follow-up.

References

- **Backup scripts:** /home/vijay/scripts/pberpprod_backup.sh, qa_backup.sh, dev_backup.sh
- **MEMORY.md:** backup cron schedule, container names, offsite destinations
- **Lessons cited:** #5 (secrets in .env), #48 (8-phase test), #77 (atomic crontab), #79 (hardened backup), #91 (image prune), #93 (ssh key chmod), #114 (silent cron failures), #151 (fixtures declaration), #153 (unicorn preload)
- **Architecture:** 00.2 Architecture Overview
- **Daily ops:** 04.2 Daily Ops
- **Incident response:** 04.4 Incident Response

Backup tested quarterly beats backup never tested. Schedule the drill.